Remix: do React Router à reinvenção da web full stack

Trabalho Prático – Frameworks: do Vanilla ao Modernismo IFFar – Campus Uruguaiana · Curso Técnico em Informática Integrantes: *Yuri Pavão, Murilo Jaques, João Vitor e Octávio Bermudez*.
Framework pesquisado: **Remix** (versão atual: Remix 3, release candidate 3.0.0-rc.1, publicada em 31/08/2026)

Resumo

O Remix é um framework web full stack criado por Ryan Florence e **Michael Jackson**, os autores do React Router. Nasceu com uma proposta simples e teimosa: usar a plataforma web (HTTP, formulários, Request e Response) em vez de esconder tudo atrás de abstrações do JavaScript. Sua história tem uma reviravolta rara: as ideias do Remix foram absorvidas pelo React Router v7, e o nome "Remix" passou a designar um framework novo, reescrito do zero e sem React como base. Este artigo percorre o problema que motivou o Remix, sua história, sua arquitetura, curiosidades, seus usos no mundo real e uma avaliação crítica do que vem pela frente.

1. Introdução

O Remix é um framework web full stack criado por Ryan Florence e Michael Jackson, também responsáveis pelo React Router. Sua proposta é aproveitar os padrões nativos da web, como HTTP, formulários e as APIs Request e Response, reduzindo a necessidade de abstrações e código JavaScript no navegador. Ele surgiu como resposta a problemas comuns das aplicações de página única (SPA), como carregamento inicial lento, requisições em cascata, tratamento repetitivo de erros e dependência excessiva de JavaScript.

2. História e evolução

O Remix foi anunciado em 2020 e lançado como código aberto em 2021. Em 2022, foi adquirido pela Shopify, que o utilizou como base para o Hydrogen, ferramenta voltada à criação de lojas virtuais personalizadas. O Remix v2 adotou o Vite e o roteamento baseado em arquivos. Em 2024, seus recursos foram incorporados ao React Router v7, no chamado “modo framework”. Depois, o nome Remix foi reutilizado para o Remix 3, uma reescrita com arquitetura própria e sem React como base. Em 31 de agosto de 2026, essa versão estava em release candidate, portanto ainda podia sofrer mudanças.

3. Arquitetura e principais conceitos

Um dos conceitos centrais do Remix clássico é o roteamento aninhado: cada parte da URL corresponde a uma camada da interface, com seus próprios dados e tratamento de erros. Ao navegar, apenas as partes necessárias são atualizadas, e os dados das rotas podem ser buscados em paralelo, reduzindo esperas desnecessárias.

As rotas podem definir duas funções executadas no servidor. O loader obtém dados para exibir uma página, normalmente em requisições de leitura (GET). A action processa alterações enviadas pelo usuário, como dados de formulários (POST, PUT ou DELETE). Depois que uma action é executada, os dados podem ser carregados novamente para atualizar a interface sem que o programador precise controlar manualmente todo o estado.

O framework também favorece a melhoria progressiva: formulários HTML continuam funcionando mesmo se o JavaScript estiver desativado ou falhar; quando ele está disponível, o envio pode ser tratado sem recarregar a página inteira. A renderização no servidor (SSR) envia HTML pronto, favorecendo a exibição inicial e o SEO. Limites de erro por rota permitem que uma falha afete apenas uma parte da tela, enquanto o streaming possibilita mostrar o conteúdo inicial antes da chegada de dados mais lentos.

4. O que muda no Remix 3

O Remix 3 mantém a ideia de construir sobre padrões da web, mas abandona o React e apresenta seu próprio modelo de componentes. Foi concebido como um conjunto de módulos que podem funcionar juntos ou separadamente, sem depender de um bundler externo. Utiliza APIs padrão, como Request e Response, buscando facilitar a execução em ambientes diferentes, como Node, Deno, Bun e plataformas edge. Também prevê ferramentas de linha de comando para tarefas com banco de dados, como migrações, seeds, reset e rollback. A equipe ainda destaca convenções previsíveis para facilitar o trabalho de desenvolvedores e assistentes de programação com IA.

5. Curiosidades e uso no mundo real

A trajetória do Remix é incomum porque parte de suas ideias foi incorporada à ferramenta de roteamento da qual ele surgiu, o React Router, enquanto o nome passou a identificar um framework diferente. Além disso, o projeto começou como produto pago antes de se tornar open source e adotou o Vite, ferramenta criada em outro ecossistema.

A Shopify é o principal exemplo de uso citado no material: o Remix está ligado ao site Shopify.com, ao Shop.app e ao Hydrogen, voltado a lojas headless. Entre as razões para escolher essa abordagem estão o carregamento mais rápido, a redução de JavaScript enviado ao navegador, o funcionamento mais resiliente quando scripts falham e a possibilidade de implantar aplicações em diferentes provedores de hospedagem.

6. Conclusão

O principal mérito do Remix é recolocar o servidor, o HTTP e os formulários no centro do desenvolvimento web. Rotas aninhadas, loaders, actions e tratamento de erros ajudam a reduzir código repetitivo e organizar melhor as aplicações. Seu maior desafio é a identidade: hoje, o nome pode se referir ao Remix v1/v2, aos recursos integrados ao React Router e ao Remix 3. Para quem deseja continuar com React, o React Router em modo framework é o caminho indicado no material. Já o Remix 3 representa uma aposta em padrões web, portabilidade, menor dependência de ferramentas externas e integração com assistentes de IA; por ainda estar em versão candidata, é prudente aguardar uma versão estável antes de adotá-lo em projetos de produção.

Referências

• Remix. Remix 3 is under active development. remix.run.

• Remix. Remix 3 Release Candidate (31/08/2026). remix.run/blog/remix-3-release-candidate.

• TechCrunch. Shopify acquires Remix to bolster its storefront design tools (31/10/2022).

• The New Stack. Remix takes on Next.js in battle of the React frameworks.

• Shopify. Documentação do Hydrogen e do Oxygen. shopify.dev.

• Build with Matija. Remix vs React Router: What Happened to Remix (20/08/2026).

• Peerlist. Remix 3: what's changing and why it matters.